

Formal Differential Geometry in the $\mathcal{UD}$ Calculus: Part 1

Brian Beckman

Jan 2026

Abstract

In this notebook, we extend and formalize the $\mathcal{UD}$ Calculus, a computational frontend for unambiguous, symbolic tensor calculations in the Wolfram Language. $\mathcal{UD}$ features concise, but unambiguous input like $T[\mathcal{U}[\mu], \mathcal{D}[\nu]]$ (“T, up μ , down ν ”) and typeset output like T^μ_ν .

We take the view that Wolfram’s existing *Symbolic Tensors* plus existing repository functions like *MetricTensor* and *RiemannTensor* furnish powerful, *backend* tools for differential geometry and relativity. $\mathcal{UD}$ on the other hand, focuses on structural logic of tensor algebra, thus a *frontend* with syntactic and semantic capabilities:

- valence checking (type safety)
- automatic index canonicalization and coalescing
- collision-free coordinate transformations

$\mathcal{UD}$ should be useful for applications of differential geometry beyond relativity. Differential geometry is enjoying rapidly growing utility in computer vision and robotics, and has long been central to classical and quantum mechanics, to field theory, and to gauge theory. We have expressed interest in these applications in many earlier notebooks and plan to revisit some of them, buttressing prior work with formal rigour via $\mathcal{UD}$.

That said, this notebook is the first in a series dedicated specifically to general relativity by implementing Eric Poisson’s *A Relativist’s Toolkit* in $\mathcal{UD}$. We will verify his derivations *ab initio*, assuring not just correctness, but rigour of the derivation. In the next installment (in preparation), we’ll cover covariant derivatives and the connections for general relativity. After that, we’ll cover geodesics, Lie derivatives, Killing vectors, the Levi-Cevita tensor, geodesic deviation, Fermi Normal Coordinates, and reproduce famous metrics like Schwarzschild and Kerr.

All along, we’ll incrementally develop, a $\mathcal{UD}$ compiler backend: a translation layer to map $\mathcal{UD}$ expressions onto Wolfram’s existing *SymbolicTensor* and elements of the function repository. Our objective is that users can derive physical laws in the high-assurance $\mathcal{UD}$ environment and then seamlessly compile them to Wolfram’s computational infrastructure.

$\mathcal{UD}$ is an exercise in formal (machine-checked) mathematics with compiler-like architecture. While the Wolfram language is not, *prima facie*, a purely formal system like Rocq, Lean, or TLA+, Wolfram’s

symbolic algebra lets us have many formal checks, not to mention numerical agility, throughout any work.

Introduction

Wolfram Mathematica is a superpower for general-purpose algebra and calculus. The Wolfram Function Repository offers user-contributed, component-oriented tools for `MetricTensor`, `ChristoffelSymbol`, `EinsteinTensor`, `SolveEinsteinEquations`, and so on, for relativity problems.

For physicists who need to understand or reproduce derivations—the index raising, lowering, and contracting, $\mathcal{UD}$ furnishes the complementary and layered frontend services:

- Grammar: $\mathcal{UD}$ knows that adding a vector to a covector is physically meaningless and will stop with a valence check.
- Notation: $\mathcal{UD}$ has easy input forms for abstractly indexed tensors. These input forms as small as possible while being unambiguous, e.g., $\mathcal{T}[\mathcal{u}[\mu], \mathcal{D}[\nu]]$. $\mathcal{UD}$'s output forms match typeset notebooks and papers as closely as possible, e.g., $\mathcal{T}^{\mu}_{\nu}$.
- Abstractions: $\mathcal{UD}$ manipulates tensors as whole symbolic objects, rather than as decorated arrays, preserving index notation in both input and output forms.
- Rigour: $\mathcal{UD}$ handles renaming of contraction-bound (dummy) indices via alpha-conversion, preventing the index-capture errors that plague pen-and-paper derivations.
- Bridges: $\mathcal{UD}$ will map to Wolfram's existing support for tensors as a target virtual machine.

Our guide for this journey is Eric Poisson's text, *A Relativist's Toolkit*. We will proceed section by section over several notebooks, implementing his textual toolkit as a software toolkit.

Notes

This work builds upon my previous notebooks on special and general relativity, where $\mathcal{UD}$ ("Up/-Down") notation was first introduced to disambiguate covariant and contravariant indices.

Deployment

To ensure this notebook is self-contained and reproducible, the entire source code for $\mathcal{UD}$'s relativity toolkit, with its unit tests and regression tests, is included inline here. In future installments of this series, as the codebase grows, we will transition to open-source distribution hosted on GitHub.

The Code and Tests

This code is evolving. This notebook contains an early version, formatted inline for human reading. You may also be interested in the latest version. We explain how to download and run it below.

Optional: Download and Run Latest version

The most up-to-date version is hosted on GitHub. This notebook contains a preliminary version inline for your inspection. To download and run the most current version, unlock and run the following cells.

They're marked "unevaluable" by default so that they won't interfere with those who want to evaluate the whole notebook. Unlock them via the "Cell" menu, "Properties" submenu, "Evaluatable" item. Expect to see unit tests and the following visual gallery.

```
In[*]:= (*1. Clean the kernel to ensure we aren't seeing cached definitions*)
Quit[]

(*2. LOAD THE CODE*)
Module[{base, rules, tests, viz, bust},
  base = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/main/";
  bust = "?t=" <> ToString[Round[AbsoluteTime[]]];
  (*Force fresh download*)
  (*Define URLs*)
  rules = base <> "RelativityToolkit.wl" <> bust;
  tests = base <> "RelativityToolkit_RegressionTests.wl" <> bust;
  viz = base <> "RelativityToolkit_VisualShowcase.wl" <> bust;
  (*Load them*)
  Get[rules];
  Get[tests];
  Get[viz](*no semicolon on purpose to see gallery*)]
```

Out[*]=

Description	Rendered Output
Generic Mixed Partial	$\frac{\partial^2 f}{\partial x \partial y}$
Coordinate Hessian (The 'Garbage Term')	$\frac{\partial^2 x^\alpha}{\partial x^\alpha \partial x^\beta}$
Nested in an Equation	$\text{lhs} = \frac{\partial^2 A^\mu}{\partial x^\lambda \partial x^\nu}$
Multiplied by Jacobian	$\frac{\partial^2 x^\alpha}{\partial x^\alpha \partial x^\beta} \frac{\partial x^\beta}{\partial x^{\beta'}}$
Comma Notation	$A^\mu_{,\nu}$
Covariant Derivative (Abstract)	$A^\mu_{;\nu}$
Covariant Derivative (Expanded)	$A^\mu_{,\nu} + (A^\lambda{}^\mu) \Gamma^\mu_{\nu\lambda}$

How to Read this Notebook

The code is presented here in reverse of the best order for human reading, because it's necessary to evaluate the implementation before showing what it can do and how to use it (*def before ref*, the standard dilemma of *Literate Programming*, which we can't practice).

The best order to read is bottom-up:

- **Visual Showcase:** A gallery showing what $\mathcal{UD}$ can do
- **Tests:** Bite-sized samples of how to use $\mathcal{UD}$
- **Implementation:** If you care how $\mathcal{UD}$ works

The code is formatted in .wl block style, with **essential prose in comments instead of in Notebook cells**, for easier upload to code repositories. *Read the comments!*

Implementation

```
In[*]:= (* =====*)
(* RELATIVITY TOOLKIT: PART 1 *)
(* Formal Differential Geometry in the UD Calculus *)
(* =====*)

(* =====*)
(* 1. CLEAN SLATE -----*)
(* =====*)

(* Clear previous definitions to ensure a fresh environment. *)
Unprotect[MakeBoxes];
Quiet[MakeBoxes[Partials[_], _], StandardForm] =.;
Quiet[MakeBoxes[ $\delta$ [__], StandardForm] =.];
Quiet[DownValues[MakeBoxes] =
  Select[
    DownValues[MakeBoxes], FreeQ[#,  $\mathcal{U}$ ] && FreeQ[#,  $\mathcal{D}$ ] &];
Protect[MakeBoxes];

ClearAll[
  valence,
  CanonicalizeIndices,
  TensorForm,
  Partials,
  upQ,
  downQ, x, p, g, u, A, B, T, P, Q, S,
  metricRules,
  simpleCommaRules,
  robustTransformRules];

(* =====*)
(* 2. VALENCE LOGIC (The Type Checker) *)
(* =====*)
```

```

(*---Predicates-----*)
upQ[x_] := upQ[valence[x]];
downQ[x_] := downQ[valence[x]];

upQ[{{_}, {}]} := True;
upQ[___] := False;

downQ[{{}}, {_}] := True;
downQ[___] := False;

(*---Helpers-----*)
contractValence[{u_List, d_List}] :=
Module[{common},
  common = Intersection[u, d];
  {DeleteElements[u, common],
   DeleteElements[d, common]};

(*---Base Cases-----*)
valence[x_Symbol] := {{}}, {};
valence[_?NumericQ] := {{}}, {};
valence[] := Null;

(*---Tensor Atoms-----*)
(*Direct matching on Global symbols for inline usage*)
valence[_[U[α_]]] := {{α}, {}};
valence[_[D[α_]]] := {{}}, {α};

(*---Metric& Delta-----*)
(*g and δ are special global symbols for the metric and Kronecker delta *)
valence[g[D[μ_], D[ν_]]] := {{}}, {μ, ν};
valence[g[U[μ_], U[ν_]]] := {{μ, ν}, {}};
valence[δ[U[μ_], D[ν_]]] := {{μ}, {ν}};

(*---Partials (Jacobians)-----*)
valence[Partials[num_, den_]] :=
Module[{vn, vd, vdFlipped},
  vn = valence[num];
  vd = valence[den];
  (*Denominator variance flips:Up→Down,Down→Up*)
  vdFlipped = {vd[[2]], vd[[1]]};
  {Join[vn[[1]], vdFlipped[[1]], Join[vn[[2]], vdFlipped[[2]]]};

(*---Differentiation-----*)
(*Gradient Rule:d(T)/dx^u flips the valence of x^u from Up to Down*)

```

```

valence[Derivative[1][f_][arg_]] :=
  Module[{u, d},
    {u, d} = valence[arg];
    If[{u, d} === {{}, {}},
      {{}, {}},
      {d, u}]];

(*Higher derivatives pass through if the function has valence*)
valence[Derivative[_][_][f_] /; (valence[f] != {{}, {}})] := valence[f];

(*---Recursion & Arithmetic-----*)
(*Pass-through for Heads like x[U[mu]]*)
valence[(h : _[u_] | _[d_])[_]] := valence[h];

valence[prod_Times] := contractValence[
  Fold[
    Join[#1, valence[#2], 2] &,
    {{}, {}},
    List@@prod]];

valence[sum_Plus] :=
  Module[{vs},
    vs = valence /@ (List@@sum);
    If[Apply[SameQ, vs],
      First[vs],
      Print["Valence Mismatch"]; {{}, {}}}];

valence[Power[b_, _]] := valence[b];

(*Multi-index recursion:T[a,b]→Join valences*)
valence[h_[a_, b_, rest___]] :=
  Join[
    valence[h[a]],
    valence[h[b, rest]], 2];

(*Fallback*)
valence[___] := {{}, {}];

(* =====*)
(* 3. DISPLAY RULES (Textbook Formatting)*)
(* =====*)

Unprotect[MakeBoxes];

```

```

(*---Partials-----*)
MakeBoxes[Partials[num_, den_], StandardForm] :=
  FractionBox[
    RowBox[{"∂", MakeBoxes[num, StandardForm]}],
    RowBox[{"∂", MakeBoxes[den, StandardForm]}]];

(*---Kronecker Delta-----*)
(*Enforce vertical stacking for Delta indices*)
MakeBoxes[δ[U[up_], D[down_]], StandardForm] :=
  SubsuperscriptBox["δ",
    MakeBoxes[down, StandardForm],
    MakeBoxes[up, StandardForm]];

MakeBoxes[δ[D[down_], U[up_]], StandardForm] :=
  SubsuperscriptBox["δ",
    MakeBoxes[down, StandardForm],
    MakeBoxes[up, StandardForm]];

(*---Generic Tensors-----*)
(*Matches ANY head h_[indices] where indices are wrapped in U[...] or D[...]*)
MakeBoxes[h_[indices_], StandardForm] /;
  (h != δ) && (h != Partials) && MatchQ[{indices}, {(U[_] | D[_]) ..}] :=
  Module[{formattedScripts},
    formattedScripts = {indices} /. {
      U[i_] → SuperscriptBox["", MakeBoxes[i, StandardForm]],
      D[i_] → SubscriptBox["", MakeBoxes[i, StandardForm]]};
    RowBox[Prepend[formattedScripts, MakeBoxes[h, StandardForm]]]]

Protect[MakeBoxes];

(* =====*)
(* 4. ALGEBRAIC SIMPLIFICATION (Index Canonicalization) *)
(* =====*)

CanonicalizeTerm[term_] :=
  Module[{indices, counts, dummies, replacementRules},
    (*Identify all Up/Down indices*)
    indices = Cases[term, (U | D)[i_] → i, Infinity];
    (*Find pairs*)
    counts = Tally[indices];
    dummies = Select[counts, Last[#] == 2 &] [[All, 1]];
    (*Create rules: mu → FormalI1, nu → FormalI2, etc.*)
    replacementRules =

```

```
MapIndexed[#1 → Symbol["i" <> ToString[First[#2]]] &, dummies];
term /. replacementRules];
```

```
CanonicalizeIndices[expr_Plus] := Total[CanonicalizeIndices /@ List @@ expr];
CanonicalizeIndices[expr_Equal] := Equal @@ (CanonicalizeIndices /@ List @@ expr);
CanonicalizeIndices[expr_] := CanonicalizeTerm[expr];
```

```
(* =====*)
(* 5. PHYSICS RULES (Metric & Transformations)*)
(* =====*)
```

```
metricRules = {
```

```
(*---LOWERING RULES (g_uv A^v→A_u)---*)
(*Standard:Contract the second index (g_uv A^v)*)
g[D[μ_], D[v_]] * vec_[U[v_]] ⇨ vec[D[μ]],
vec_[U[v_]] * g[D[μ_], D[v_]] ⇨ vec[D[μ]],
(*Symmetry:Contract the first index (g_vu A^v)*)
g[D[v_], D[μ_]] * vec_[U[v_]] ⇨ vec[D[μ]],
vec_[U[v_]] * g[D[v_], D[μ_]] ⇨ vec[D[μ]],

(*---RAISING RULES (g^uv A_v→A^u)---*)
(*Standard:Contract the second index (g^uv A_v)*)
g[U[μ_], U[v_]] * covec_[D[v_]] ⇨ covec[U[μ]],
covec_[D[v_]] * g[U[μ_], U[v_]] ⇨ covec[U[μ]],
(*Symmetry:Contract the first index (g^vu A_v)*)
g[U[v_], U[μ_]] * covec_[D[v_]] ⇨ covec[U[μ]],
covec_[D[v_]] * g[U[v_], U[μ_]] ⇨ covec[U[μ]],

(*---INVERSE IDENTITY (g^ua g_av→delta^u_v)---*)
g[U[μ_], U[α_]] * g[D[α_], D[v_]] ⇨ δ[U[μ], D[v]],
g[D[α_], D[v_]] * g[U[μ_], U[α_]] ⇨ δ[U[μ], D[v]]};
```

```
robustTransformRules = {
```

```
(*Vector Transformation with Alpha-Conversion*)
A_[U[primedIndex_]] ⇨
Module[{freshIndex},
  freshIndex = Unique["μ"];
  Partials[x[U[primedIndex]], x[U[freshIndex]]] * A[U[freshIndex]],
(*Covector Transformation with Alpha-Conversion*)
p_[D[primedIndex_]] ⇨
Module[{freshIndex},
  freshIndex = Unique["ν"];
```



```

Partials[x[U[freshIndex]], x[U[primedIndex]] * p[D[freshIndex]]];

(* =====*)
(* 6. PRETTY PRINTING (TensorForm)*)
(* =====*)

TensorForm[expr_] :=
Module[{canonExpr, indexMap, prettyIndices},
  canonExpr = CanonicalizeIndices[expr];
  (*Standard pool of physics indices for display*)
  prettyIndices = {λ, κ, ρ, σ, μ, ν, τ, η, χ, ψ};
  indexMap = Table[
    Symbol["i" <> ToString[n]] → prettyIndices[[n]], {n, Length[prettyIndices]};
  HoldForm[Evaluate[canonExpr /. indexMap]]];

Print["Relativity Toolkit Part 1 (Inline) Loaded Successfully."];
Relativity Toolkit Part 1 (Inline) Loaded Successfully.

```

Smoke Tests

```

In[*]:= Print["--- 1. Visualizing Tensors ---"];
(*We input unambiguous structure:x[U[mu]]*)
(*The system renders textbook notation:*)
{"Vector (Contravariant)" → x[U[μ]],
 "Covector (Covariant)" → p[D[ν]],
 "Mixed Tensor T^u_v" → T[U[μ], D[ν]],
 "Mixed Tensor T_u^v" → T[D[μ], U[ν]]} // TableForm

--- 1. Visualizing Tensors ---

Out[*]//TableForm=
Vector (Contravariant) → x μ
Covector (Covariant) → p ν
Mixed Tensor T^u_v → T μ ν
Mixed Tensor T_u^v → T μ ν

```

```

In[*]:= Print["--- PHYSICS ENGINE DIAGNOSTICS ---"];

(*1. METRIC RAISING & LOWERING*)
Print["\n1. Metric Gymnastics:"];
termLower = g[D[μ], D[v]] × A[U[v]];
resLower = termLower /. metricRules;
Echo[termLower, "Input g[D[μ],D[v]] A[U[v]]: "];
Echo[resLower, "Result (A_u): "];

termInverse = g[U[μ], U[α]] × g[D[α], D[v]];
resInverse = termInverse /. metricRules;
Echo[termInverse, "Metric Product: g[U[μ],U[α]] g[D[α],D[v]]"];
Echo[resInverse, "Inverse Identity (Delta^u_v): "];

(*2. VALENCE CHECKER*)
Print["\n2. Valence Safety Check:"];
(*Should be True*)
valCheck = (valence[resLower] === {{}, {μ}});
Echo[valCheck, "Did A^v become A_u? "];

(*Should be False/Error for bad physics*)
badPhysics = x[U[μ]] + p[D[μ]];
Print["Checking Bad Physics (Expect Error Message below):"];
valence[badPhysics];

(*3. ALPHA-CONVERSION (Collision Test)*)
Print["\n3. Coordinate Transformation (Alpha-Conversion):"];
(*Transform A^a' → (dx^a'/dx^mu) A^mu*)
(*Watch for Unique indices like mu123*)
contraTrans = A[U[Superscript["a", ","]]];
contraResTrans = contraTrans /. robustTransformRules;

Echo[contraTrans, "Original Contravariant: "];
Echo[contraResTrans, "Transformed (Look for Unique Indices): "];

covTrans = A[D[Superscript["a", ","]]];
covResTrans = covTrans /. robustTransformRules;

Echo[covTrans, "Original Covariant: "];
Echo[covResTrans, "Transformed (Look for Unique Indices): "];

```

--- PHYSICS ENGINE DIAGNOSTICS ---

1. Metric Gymnastics:

- » Input $g[\mathcal{D}[\mu], \mathcal{D}[v]] A[\mathcal{U}[v]]$: $(A^\nu) (g_{\mu\nu})$
- » Result (A_u): A_μ
- » Metric Product: $g[\mathcal{U}[\mu], \mathcal{U}[\alpha]] g[\mathcal{D}[\alpha], \mathcal{D}[v]]$ $(g_{\alpha\nu}) (g^{\mu\alpha})$
- » Inverse Identity (Delta^u_v): δ^μ_ν

2. Valence Safety Check:

- » Did A^v become A_u? True

Checking Bad Physics (Expect Error Message below):

Valence Mismatch

3. Coordinate Transformation (Alpha-Conversion):

- » Original Contravariant: $A^{a'}$
- » Transformed (Look for Unique Indices): $(A^{\mu 11}) \frac{\partial x^{a'}}{\partial x^{\mu 11}}$
- » Original Covariant: $A_{a'}$
- » Transformed (Look for Unique Indices): $(A_{\nu 12}) \frac{\partial x^{\nu 12}}{\partial x^{a'}}$

Regression Tests

```
In[*]:= (* =====*)
(* RELATIVITY TOOLKIT PART 1: REGRESSION SUITE*)
(* =====*)

Print["\n====="];
Print["RUNNING REGRESSION SUITE: THE VECTOR SPACE & METRIC"];
Print["=====\n"];

(*---TEST HELPER FUNCTIONS-----*)
PassCount = 0;
FailCount = 0;

AssertValence[expr_, expected_, label_] :=
Module[{v = valence[expr]},
If[v == expected,
PassCount++;
Print["[PASS] ", label, " -> ", v],
FailCount++;
```

```

Print["[FAIL] ", label, " \n\tExpected: ", expected, "\n\tGot:      ", v]]];

AssertEqual[expr1_, expr2_, label_] :=
Module[{c1 = CanonicalizeIndices[expr1], c2 = CanonicalizeIndices[expr2]},
  If[c1 === c2,
    PassCount++;
    Print["[PASS] ", label],
    FailCount++;
    Print["[FAIL] ", label, " \n\tLHS: ", c1, "\n\tRHS: ", c2]]];

(* =====*)
(* SECTION 1: TYPE CHECKING (VALENCE) *)
(* =====*)
Print["--- SECTION 1: TYPE CHECKING (VALENCE) ---"];

(*1. Atoms*)
AssertValence[x[U[μ]], {{μ}, {}}, "Vector x^u"];
AssertValence[p[D[v]], {{}, {v}}, "Covector p_v"];
AssertValence[T[U[μ], D[v]], {{μ}, {v}}, "Mixed Tensor T^u_v"];
AssertValence[T[D[μ], D[v]], {{}, {μ, v}}, "Mixed Tensor T_uv"];
AssertValence[T[U[μ], U[v]], {{μ, v}, {}}, "Mixed Tensor T^uv"];
AssertValence[T[D[μ], U[v]], {{v}, {μ}}, "Mixed Tensor T_u^v"];

(*2. Contractions*)
AssertValence[x[U[μ]] × p[D[μ]], {{}, {}}, "Scalar Contraction x^u p_u"];
AssertValence[p[D[μ]] × x[U[μ]], {{}, {}}, "Scalar Contraction p_u x^u"];
AssertValence[T[U[μ], D[v]] × x[U[v]], {{μ}, {}}, "Tensor-Vector T^u_v x^v -> x^u"];
AssertValence[T[U[μ], D[v]] × x[D[μ]], {{}, {v}}, "Tensor-Vector T^u_v x_u -> x_n"];

(*3. Arithmetic*)
AssertValence[(x[U[μ]] × p[D[μ]])^2, {{}, {}}, "Power of Scalar"];
AssertValence[5 × x[U[μ]], {{μ}, {}}, "Scalar Multiplication"];

(*4. Differentiation (The Gradient Test)*)
(*d(Scalar)/dx^u→Down Index u*)
AssertValence[
  Derivative[1][f][x[U[α]]], {{}, {α}},
  "Gradient d(f)/dx^a -> Down[a]"];

(*5. Error Handling*)
Quiet[Module[{badSum = x[U[μ]] + p[D[μ]]},
  If[valence[badSum] === {{}, {}},
    PassCount++;
    Print["[PASS] Mismatch Detection (Handled Gracefully)"],

```

```

    FailCount++;
    Print["[FAIL] Mismatch Detection failed"]]]];

(* =====*)
(* SECTION 2: ALGEBRA (CANONICALIZATION) *)
(* =====*)
Print["\n--- SECTION 2: ALGEBRA (CANONICALIZATION) ---"];

(*1. Dummy Renaming*)
AssertEqual[
  x[u[mu]] x[p[d[mu]]],
  x[u[v]] x[p[d[v]]],
  "Dummy Renaming ( $A^u B_u == A^v B_v$ )"];

(*2. Commutativity*)
AssertEqual[
  x[u[mu]] x[p[d[mu]]],
  p[d[v]] x[u[v]],
  "Commutativity ( $A^u B_u == B_v A^v$ )"];

(*3. Index Coalescing (Summing identical terms)*)
AssertEqual[
  x[u[mu]] x[p[d[mu]]] + x[u[v]] x[p[d[v]]],
  2 * x[u[rho]] x[p[d[rho]]],
  "Index Coalescing (Summing identical terms)"];

(*4. Double-Dummy Pairs*)
AssertEqual[
  P[u[mu]] x[Q[d[mu]]] x[S[u[v]]] x[T[d[v]]],
  S[u[alpha]] x[T[d[alpha]]] x[P[u[beta]]] x[Q[d[beta]]],
  "Double Dummy Pairs ( $P^u Q_u S^v T_v$ )"];

(*5. Negative Test*)
If[CanonicalizeIndices[A[u[alpha]] x[B[u[beta]]] !=
  CanonicalizeIndices[A[u[mu]] x[B[u[mu]]],
  PassCount++;
  Print["[PASS] Free Indices Preserved (Distinct indices do not merge)"],
  FailCount++;
  Print["[FAIL] Free Indices Incorrectly Merged"]];

(* =====*)
(* SECTION 3: PHYSICS (METRIC & TRANSFORMATIONS) *)
(* =====*)
Print["\n--- SECTION 3: PHYSICS (METRIC & TRANSFORMATIONS) ---"];

```

```

(*1. Raising & Lowering*)
AssertEqual[
  (g[D[μ], D[ν]] × A[U[ν]]) /. metricRules,
  A[D[μ]],
  "Lowering Index (g_uv A^v -> A_u)"];

AssertEqual[
  (g[U[μ], U[ν]] × p[D[ν]]) /. metricRules,
  p[U[μ]],
  "Raising Index (g^uv p_v -> p^u)"];

(*2. Inverse Identity*)
AssertEqual[
  (g[U[μ], U[α]] × g[D[α], D[ν]]) /. metricRules,
  δ[U[μ], D[ν]],
  "Inverse Metric (g^ua g_av -> delta^u_v)"];

(*3. Alpha-Conversion (Collision Safety)*)
Module[{t1, t2, res, dummies, unique},
  t1 = A[U[Superscript["a", "/"]]];
  t2 = B[U[Superscript["b", "/"]]];
  res = (t1*t2) /. robustTransformRules;
  (*Find ANY Symbol inside U or D. Since input was Superscript,
  these MUST be generated indices.*)
  dummies = Cases[res, (U | D)[i_Symbol] :> i, Infinity];
  unique = DeleteDuplicates[dummies];
  (*Expect exactly 2 distinct indices (one for A, one for B)*)
  If[Length[unique] == 2,
    PassCount++;
    Print["[PASS] Alpha-Conversion (Generated distinct indices: ", unique, ")"],
    FailCount++;
    Print["[FAIL] Alpha-Conversion (Collision detected: ", unique, ")"]];];

(* =====*)
(* METRIC EQUIVALENCE TEST (The Physicality Check) *)
(* =====*)
Print["\n--- METRIC EQUIVALENCE (A^u B_u == A_v B^v) ---"];

(*1. Define the two forms*)
termUp = A[U[μ]] × B[D[μ]]; (*A^u B_u*)
termDown = A[D[ν]] × B[U[ν]]; (*A_v B^v*)

(*2. Verify they are STRUCTURALLY distinct*)

```

```

If[CanonicalizeIndices[termUp] != CanonicalizeIndices[termDown],
  PassCount++;
  Print["[PASS] Structural Distinction ( $A^u B_u \neq A_v B^v$  initially)"],
  FailCount++;
  Print["[FAIL] Structural Distinction failed"]];

(*3. Verify they are PHYSICALLY equivalent*)
(*PROOF STRATEGY:
  1. Expand  $A_v \rightarrow g_{va} A^a$ .
  2. Hold[]  $A^a$  so the metric doesn't immediately re-lower it.
  3, forcing forces the metric to lower B instead.*)

expandRule = A[D[idx_]] :>
  Module[{fresh = Unique["a"]},
    g[D[idx], D[fresh]] * Hold[A[U[fresh]]]; (*solve contraction "race"*)

(*Step A:Expand and Freeze A*)
termExpanded = termDown /. expandRule;

(*Step B:Apply Metric Rules (Metric can only see B,not A)*)
termReduced = termExpanded /. metricRules;

(*Step C:Unfreeze A*)
termFinal = ReleaseHold[termReduced];

(*Step D:Compare*)
AssertEqual[termFinal, termUp,
  "Metric Equivalence ( $A_v B^v$  reduces to  $A^u B_u$ )"];

(* ===== *)
(* SUMMARY *)
(* ===== *)
Print["\n-----"];
Print["REGRESSION SUITE COMPLETE"];
Print["PASSED: ", PassCount];
Print["FAILED: ", FailCount];
If[FailCount == 0,
  Print["STATUS: GREEN (Ready for Publication)"],
  Print["STATUS: RED (Fix bugs before publishing)"]];
Print["-----"];

=====
RUNNING REGRESSION SUITE: THE VECTOR SPACE & METRIC

```

=====

--- SECTION 1: TYPE CHECKING (VALENCE) ---

```
[PASS] Vector x^u -> {{μ}, {}}
```

```
[PASS] Covector p_v -> {{}, {ν}}
```

```
[PASS] Mixed Tensor T^u_v -> {{μ}, {ν}}
```

```
[PASS] Mixed Tensor T_uv -> {{}, {μ, ν}}
```

```
[PASS] Mixed Tensor T^uv -> {{μ, ν}, {}}
```

```
[PASS] Mixed Tensor T_u^v -> {{ν}, {μ}}
```

```
[PASS] Scalar Contraction x^u p_u -> {{}, {}}
```

```
[PASS] Scalar Contraction p_u x^u -> {{}, {}}
```

```
[PASS] Tensor-Vector T^u_v x^v -> x^u -> {{μ}, {}}
```

```
[PASS] Tensor-Vector T^u_v x_u -> x_n -> {{}, {ν}}
```

```
[PASS] Power of Scalar -> {{}, {}}
```

```
[PASS] Scalar Multiplication -> {{μ}, {}}
```

```
[PASS] Gradient d(f)/dx^a -> Down[a] -> {{}, {α}}
```

Valence Mismatch

```
[PASS] Mismatch Detection (Handled Gracefully)
```

--- SECTION 2: ALGEBRA (CANONICALIZATION) ---

```
[PASS] Dummy Renaming (A^u B_u == A^v B_v)
```

```
[PASS] Commutativity (A^u B_u == B_v A^v)
```

```
[PASS] Index Coalescing (Summing identical terms)
```

```
[PASS] Double Dummy Pairs (P^u Q_u S^v T_v)
```

```
[PASS] Free Indices Preserved (Distinct indices do not merge)
```

--- SECTION 3: PHYSICS (METRIC & TRANSFORMATIONS) ---

```
[PASS] Lowering Index (g_uv A^v -> A_u)
```

```
[PASS] Raising Index (g^uv p_v -> p^u)
```

```
[PASS] Inverse Metric (g^ua g_av -> delta^u_v)
```

```
[PASS] Alpha-Conversion (Generated distinct indices: {μ13, μ14})
```

--- METRIC EQUIVALENCE (A^u B_u == A_v B^v) ---

```
[PASS] Structural Distinction (A^u B_u != A_v B^v initially)
```

```
[PASS] Metric Equivalence (A_v B^v reduces to A^u B_u)
```

REGRESSION SUITE COMPLETE

PASSED: 25

FAILED: 0

STATUS: GREEN (Ready for Publication)

Visual Showcase

```
In[*]:= Print["\n====="];
Print[Style["VISUAL SHOWCASE: FORMAL DIFFERENTIAL GEOMETRY", Bold, 16]];
Print["=====\\n"];

(*---1. THE ZOO OF TENSORS-----*)
Print[Style["1. The Tensor Zoo (Standard Objects)", Bold, 14]];
Print["These objects carry valence metadata hidden inside their structure."];

tensorZoo = {
  {"Object Name", "Wolfram Input", "Textbook Output"},
  {"Contravariant Vector", "x[u[μ]]", x[u[μ]]},
  {"Covariant Covector", "p[D[v]]", p[D[v]]},
  {"Mixed Tensor", "T[u[μ], D[v]]", T[u[μ], D[v]]},
  {"The Metric", "g[D[μ], D[v]]", g[D[μ], D[v]]},
  {"Inverse Metric", "g[u[μ], u[v]]", g[u[μ], u[v]]},
  {"Kronecker Delta", "δ[u[μ], D[v]]", δ[u[μ], D[v]]},
  {"Jacobian (Partial)", "Partials[x[u[α]], x[u[β]]]",
    Partials[x[u[α]], x[u[β]]]}};

Print[Grid[tensorZoo,
  Frame → All,
  FrameStyle → Directive[Thin, GrayLevel[0.8]],
  Background → {None, {LightGray, None}},
  Spacings → {2, 1.5},
  Alignment → {{Left, Left, Center}, Center}]];

(*---2. METRIC GYMNASTICS-----*)
Print["\\n", Style["2. Metric Gymnastics (Raising & Lowering)", Bold, 14]];
Print["The metric 'swallows' indices to change their valence."];

(*Define a helper to pretty-print operations*)
VisualizeOp[name_, input_, rule_] :=
  Module[{res}, res = input /. rule;
  Print[Grid[{
```

```

        {Style[name, Bold, 12], SpanFromLeft},
        {"Input:", input},
        {"Result:", res}},
    Alignment → Left,
    Spacings → {2, 1},
    Frame → {{True, False}, {False, True, False}},
    (*Left bar*)FrameStyle → Thick]]];];

VisualizeOp["Index Lowering", g[D[μ], D[ν]] × A[U[ν]], metricRules];

VisualizeOp["Index Raising", g[U[μ], U[ν]] × p[D[ν]], metricRules];

VisualizeOp["The Inverse Identity", g[U[μ], U[α]] × g[D[α], D[ν]], metricRules];

(*---3. COORDINATE TRANSFORMATIONS-----*)
(*FIX:Use comma instead of <>*)
Print["\n", Style["3. Coordinate Transformations (Alpha-Conversion)", Bold, 14]];
Print["Automatically generated unique dummy indices (e.g., μ$...)."];

VisualizeOp["Transforming a Vector",
  A[U[Superscript["a", "/"]]],
  robustTransformRules];

(*---4. PRETTY PRINTING-----*)
(*FIX:Use comma instead of <>*)
Print["\n", Style["4. Canonicalization & Pretty Printing", Bold, 14]];
Print["Simplifying dummy indices into standard Greek letters."];

rawSum = A[U[μ]] × B[D[μ]] + A[U[ν]] × B[D[ν]];

Print[Grid[{
  {"Raw Sum (Different Indices):", rawSum},
  {"Canonicalized (TensorForm):", TensorForm[rawSum]}},
  Alignment → Left, Spacings → {2, 1.5}]];

(*---5. METRIC EQUIVALENCE DEMO -----*)
Print["\n", Style["5. Physical Equivalence Proof", Bold, 14]];
Print["Demonstrating that ",
  Style["AμBμ", Italic],
  " is equivalent to ",
  Style["AνBν", Italic],

```

```

"."];
Print["(U will reorganize and canonicalize displays.)"];

(*Define the starting point:A_v B^v*)
startTerm = A[D[v]] × B[U[v]];

(*Step 1: Definition*)
step1Term = (g[D[v], D[α]] × A[U[α]]) B[U[v]];

(*Step 2: Commutation*)
step2Term = A[U[α]] (g[D[v], D[α]] × B[U[v]]);

(*Step 3: Contraction*)
step3Term = step2Term /. metricRules;

(*Helper for styling*)
ClearAll[TableHead];
TableHead[txt_] := Style[txt, Bold, 12, FontFamily → "Helvetica"];

Print[Grid[{
  {TableHead["Step"], TableHead["Expression"], TableHead["Input"]},
  {"1. Start", startTerm, "Input: A[D[v]]B[U[v]]"},
  {"2. Definition", step1Term, "(g[D[v],D[α]] A[U[α]]) B[U[v]]"},
  {"3. Commute?", step2Term, "A[U[α]] (g[D[v],D[α]] B[U[v]])"},
  {"4. Contract", step3Term, "step2Term/. metricRules"},
  {"5. Equality?", step3Term === startTerm, "lhs:4 === rhs:1"}},
Frame → All,
FrameStyle → Directive[Thin, GrayLevel[0.7]],
Background → {None, {LightGray, None}},
Spacings → {2, 1.5},
Alignment → {{Left, Center, Left}, Center}]];

Print["\n====="];

```

VISUAL SHOWCASE: FORMAL DIFFERENTIAL GEOMETRY

1. The Tensor Zoo (Standard Objects)

These objects carry valence metadata hidden inside their structure.

Object Name	Wolfram Input	Textbook Output
Contravariant Vector	<code>x[U[μ]]</code>	x^μ
Covariant Covector	<code>p[D[v]]</code>	p_ν
Mixed Tensor	<code>T[U[μ], D[v]]</code>	T^μ_ν
The Metric	<code>g[D[μ], D[v]]</code>	$g_{\mu\nu}$
Inverse Metric	<code>g[U[μ], U[v]]</code>	$g^{\mu\nu}$
Kronecker Delta	<code>δ[U[μ], D[v]]</code>	δ^μ_ν
Jacobian (Partial)	<code>Partials[x[U[α]], x[U[β]]]</code>	$\frac{\partial x^\alpha}{\partial x^\beta}$

2. Metric Gymnastics (Raising & Lowering)

The metric 'swallows' indices to change their valence.

Index Lowering

Input: $(A^\nu) (g_{\mu\nu})$

Result: A_μ

Index Raising

Input: $(g^{\mu\nu}) (p_\nu)$

Result: p^μ

The Inverse Identity

Input: $(g_{\alpha\nu}) (g^{\mu\alpha})$

Result: δ^μ_ν

3. Coordinate Transformations (Alpha-Conversion)

Automatically generated unique dummy indices (e.g., $\mu\$...$).

Transforming a Vector

Input: $A^{a'}$

Result: $(A^{\mu16}) \frac{\partial x^{a'}}{\partial x^{\mu16}}$

4. Canonicalization & Pretty Printing

Simplifying dummy indices into standard Greek letters.

Raw Sum (Different Indices): $(A^\mu) (B_\mu) + (A^\nu) (B_\nu)$

Canonicalized (TensorForm): $2 (A^\lambda) (B_\lambda)$

5. Physical Equivalence Proof

Demonstrating that $A^\mu B_\mu$ is equivalent to $A^\nu B_\nu$.

($\mathcal{UD}$ will reorganize and canonicalize displays.)

Step	Expression	Input
1. Start	$(A^\nu) (B_\nu)$	Input: $A[\mathcal{D}[\nu]] B[\mathcal{U}[\nu]]$
2. Definition	$(A^\alpha) (B_\nu) (g_{\nu\alpha})$	$(g[\mathcal{D}[\nu], \mathcal{D}[\alpha]] A[\mathcal{U}[\alpha]]) B[\mathcal{U}[\nu]]$
3. Commute?	$(A^\alpha) (B_\nu) (g_{\nu\alpha})$	$A[\mathcal{U}[\alpha]] (g[\mathcal{D}[\nu], \mathcal{D}[\alpha]] B[\mathcal{U}[\nu]])$
4. Contract	$(A^\nu) (B_\nu)$	step2Term/. metricRules
5. Equality?	True	lhs:4 === rhs:1

=====

Conclusion

In this first part of a planned series, we build and test $\mathcal{UD}$'s foundation algebraic engine. Presaging future integration to existing component-based tools, we introduce rules and functions for the structure of differential geometry, both generically and as applied to relativity.

We have implemented:

- Valence Checking: A type system that forbids illegal tensor operations
- Canonicalization: An algebra engine that recognizes, for instance, that $A^\mu B_\mu$, $A^\nu B_\nu$, etc. are the same objects
- The Metric: An operator that automates isomorphisms resulting from raising and lowering indices, capable, for instance, of showing, that $A^\mu B_\mu = A_\mu B^\mu$ if $A_\mu = g_{\mu\nu} A^\nu$

Coming Next: Covariant Derivatives and Connections

Algebra is only part of the story. In Part 2, we will tackle Poisson's Sections 1.2 and 1.3. We will implement:

- Christoffel Symbols
- The Covariant Derivative
- The Einstein Connection that generates gravitation

The Horizon: Part 3 and Beyond

After we're done with a self-contained Relativity Toolkit, we plan to introduce The $\mathcal{UD}$ Compiler Backend. This will link our formal, high-assurance frontend structures to Wolfram's SymbolicTensor library and the many support functions in the function repository, translating abstract geometric objects into concrete component arrays, e.g., mapping a formal Schwarzschild metric to its explicit 4×4 matrix representation. We also envision expanding our *valence* type-checker to include metadata about

domains (e.g., Complex, Spinor), and dimensions for both expanded robustness and applicability, say to SU(3) gauge theory.

References

1. *Poisson, Eric. A Relativist's Toolkit: The Mathematics of Black Hole Mechanics.* Cambridge University Press, 2004.
2. *Beckman, Brian. From the steam age to quantum fields: a unified approach via UD tensorial calculus.* Wolfram Community post.
3. *Beckman, Brian. General relativity in the UD calculus.* Wolfram Community post.
4. *Wolfram Function Repository. MetricTensor, RiemannTensor, EinsteinTensor.* (For component-based calculations).
5. *Wolfram Research. Mathematica Documentation: Tensor Calculus.*